

Project Report: News Article Classification using NLP

Mohamedsipli M
Roll Number: 216215006

Spring 2016

Submitted in fulfillment of the requirements for the degree of
Master of Science (M.Sc.) in Computer Science
Department of Computer Applications
National Institute of Technology, Tiruchirappalli
for the course
Data Mining (CAS764)
under the guidance of
Prof. Ramadoss

Abstract

This report details a machine learning project focused on classifying news articles into predefined categories using Natural Language Processing (NLP) techniques. The project utilized the 20 Newsgroups dataset, a widely-used benchmark for text classification. The workflow included essential steps such as data loading, exploratory data analysis, text preprocessing (lowercasing, removal of noise, tokenization, stop word removal, and stemming), and conversion of text into numerical features using TF-IDF with n-grams. A Linear Support Vector Machine (SVM) model was trained on the extracted features. The model's performance was evaluated using metrics including accuracy, classification report (precision, recall, F1-score), and confusion matrix. Analysis was also conducted on the most common words and the most influential features learned by the model. The project demonstrates a standard and effective pipeline for text classification and provides insights into the characteristics of the dataset and the behavior of the trained model.

Contents

1	Introduction	3
2	Methodology	3
2.1	Data Loading	3
2.2	Data Analysis and Visualization	3
2.3	Preprocessing	3
2.4	Data Splitting	5
2.5	Feature Extraction	5
2.6	Model Training	5
2.7	Model Evaluation	5
3	Results and Discussion	5
3.1	Model Analysis: Top Features per Category	6
4	Conclusion	7

1 Introduction

In the era of information overload, automatically categorizing text data is a fundamental task in many applications, including content filtering, topic modeling, and information retrieval. News article classification, specifically, aims to assign news documents to one or more predefined topic categories. This project explores the application of machine learning techniques to this problem using a classic dataset.

The 20 Newsgroups dataset is a collection of approximately 18,000 newsgroup documents, split across 20 different newsgroup categories. It is a well-known benchmark for evaluating text classification algorithms due to its size and the natural variations and overlaps in language used across different discussion topics.

The primary objective of this project is to implement and evaluate a machine learning model capable of accurately classifying documents from the 20 Newsgroups dataset. This involves the standard steps of a text classification pipeline: data loading, thorough preprocessing to clean the text, feature extraction to convert text into a numerical format, training a suitable classification model, and evaluating its performance using appropriate metrics and visualizations.

This project was conducted as part of the Data Mining course (CAS764, Spring 2016) in the M.Sc. Computer Science program at NIT Tiruchirappalli, under the guidance of Prof. Ramadoss.

2 Methodology

The project followed a standard machine learning pipeline tailored for text data, as described in the following subsections.

2.1 Data Loading

The 20 Newsgroups dataset was loaded directly using the `fetch_20newsgroups` utility from the `sklearn.datasets` module. The entire dataset (`subset='all'`) was used, and noisy components such as headers, footers, and quotes were removed using the `remove=('headers', 'footers', 'quotes')` option to focus on the main content of the articles.

The dataset was loaded into Python variables `x` (list of document strings) and `y` (list of target category indices). The names of the categories were stored in `target_names`. The dataset contained 18,846 documents spread across 20 distinct categories.

2.2 Data Analysis and Visualization

Before preprocessing and modeling, exploratory data analysis was performed to understand the characteristics of the dataset:

- **Category Distribution:** The distribution of documents across the 20 categories was analyzed and visualized using a count plot. As shown in Figure 1, the distribution reveals...
- **Document Length Analysis:** The number of words in each document was calculated and visualized using a histogram. This revealed that document lengths vary significantly, with a long tail towards very long articles, as illustrated in Figure 2. The average document length was approximately 181.64 words, the median was 83.00 words, and the maximum was 11,765 words.
- **Most Common Words:** The most frequent words were analyzed both before and after applying the preprocessing steps. This analysis highlights the impact of preprocessing (like stop word removal and stemming) in transforming the word distribution from mostly generic terms to more potentially meaningful ones.

2.3 Preprocessing

Raw text data often contains noise and inconsistencies that can negatively impact model performance. The following preprocessing steps were applied to the text documents:

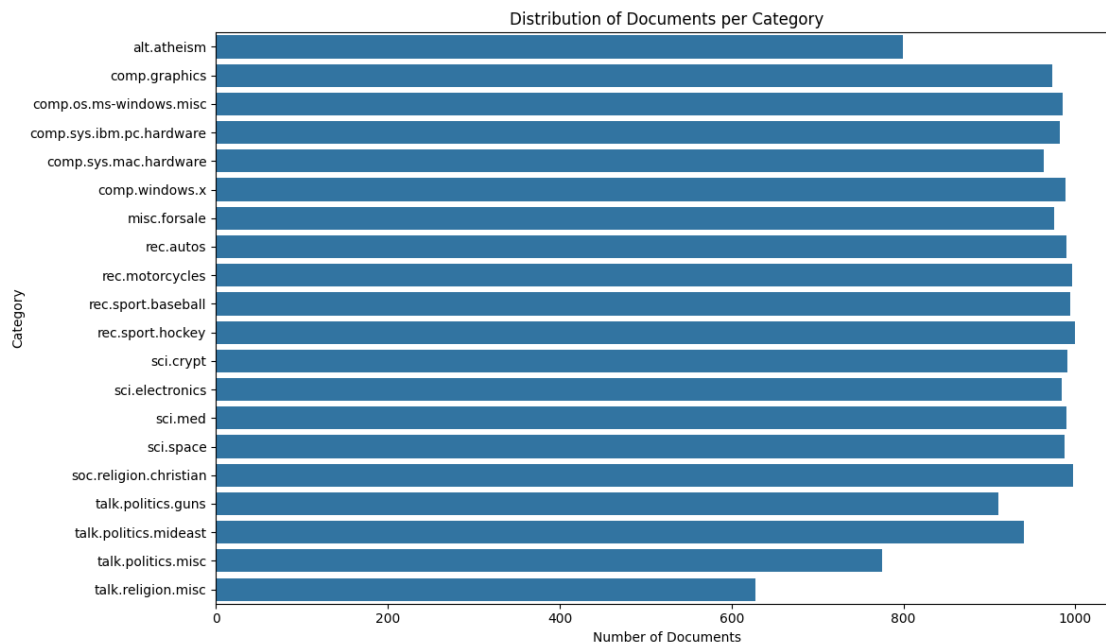


Figure 1: Distribution of Documents per Category

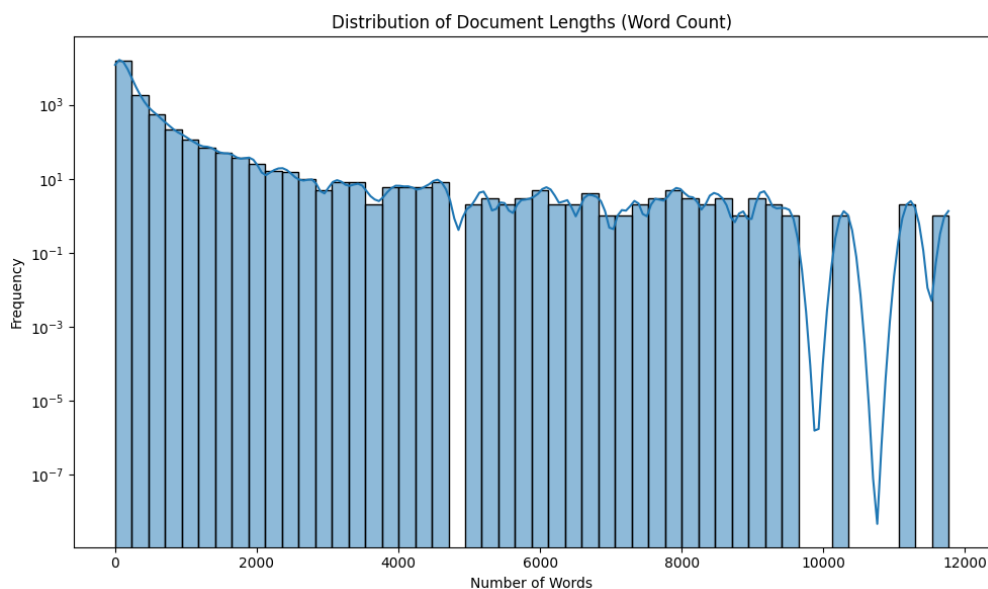


Figure 2: Distribution of Document Lengths (Word Count)

- **Lowercasing:** All text was converted to lowercase to treat words identically regardless of capitalization.
- **Removing Numbers and Punctuation:** Regular expressions were used to remove numerical digits and punctuation marks.
- **Tokenization:** The cleaned text was split into individual words or tokens using a simple split by whitespace.
- **Stop Word Removal:** Common English stop words (e.g., 'the', 'is', 'and') were removed using a list

from the NLTK library, as these words typically do not carry significant meaning for classification.

- **Stemming:** Words were reduced to their root form using the Porter Stemmer from NLTK (e.g., 'running', 'runs', 'ran' all become 'run'). This helps to group related words together.

A dedicated function `preprocess_text` was defined to apply these steps to each document.

2.4 Data Splitting

The preprocessed data (`x_processed`) and the target labels (`y`) were split into training and testing sets using `train_test_split`. A split ratio of 75% for training (14,134 documents) and 25% for testing (4,712 documents) was used. `random_state=42` ensured reproducibility, and `stratify=y` was used to maintain the original proportion of categories in both sets, which is important for multi-class datasets.

2.5 Feature Extraction

To represent the text data numerically, the TF-IDF (Term Frequency-Inverse Document Frequency) technique was used. TF-IDF assigns weights to words based on their frequency within a document relative to their frequency across the entire corpus. This helps to highlight words that are important to a specific document but not overly common everywhere.

To capture contextual information and increase the feature space's richness, n-grams were included. The `TfidfVectorizer` was configured with `ngram_range=(1, 2)`, meaning it considered both individual words (unigrams) and pairs of consecutive words (bigrams) as features. The `max_features=20000` parameter was set to limit the vocabulary size to the 20,000 most frequent unique unigrams and bigrams, resulting in a training data shape of (14134, 20000). The resulting TF-IDF matrix was highly sparse (sparsity: 0.003216), typical for text data representation.

2.6 Model Training

A Linear Support Vector Classifier (`LinearSVC`) was chosen as the classification model. Linear models, particularly Linear SVM, are well-suited for high-dimensional and sparse data like TF-IDF features due to their efficiency and effectiveness in finding linear decision boundaries in the feature space. The model was trained on the transformed training data (`x_train_tfidf`).

2.7 Model Evaluation

The trained `LinearSVC` model was evaluated on the held-out test set (`x_test_tfidf`, `y_test`). The following metrics were used to assess performance:

- **Accuracy Score:** The overall percentage of correctly classified documents.
- **Classification Report:** Provided precision, recall, and F1-score for each of the 20 categories, as well as macro and weighted averages.
- **Confusion Matrix:** A matrix showing the counts of true positives, true negatives, false positives, and false negatives for each class. A heatmap visualization of the confusion matrix was generated (see Figure 3).

3 Results and Discussion

Upon evaluating the model on the test set, the following key results were observed:

The model achieved an overall accuracy of `**0.7411**`. For a 20-class classification problem, where random chance is 5%, an accuracy of over 74% is a strong result, indicating that the model has effectively learned to distinguish between the newsgroup categories.

The Classification Report provided a detailed breakdown of performance per category. Performance varied across categories, with some being classified much more accurately (e.g., categories related to specific

software like ‘comp.windows.x’ or specific sports like ‘rec.sport.hockey’ than others (e.g., those with more general or overlapping topics like ‘alt.atheism’ or ‘talk.religion.misc’). Analyzing the precision and recall values for individual classes helps understand the types of misclassification errors occurring.

The Confusion Matrix visualization (see Figure 3) provided insight into which specific pairs of categories were most often confused by the model. Looking at the heatmap, the diagonal elements represent correct classifications, while off-diagonal elements indicate misclassifications. Clusters of errors among related categories (e.g., within the ‘comp’ groups or ‘talk.politics’ groups) would be visible, highlighting areas where the model struggles to make fine-grained distinctions.

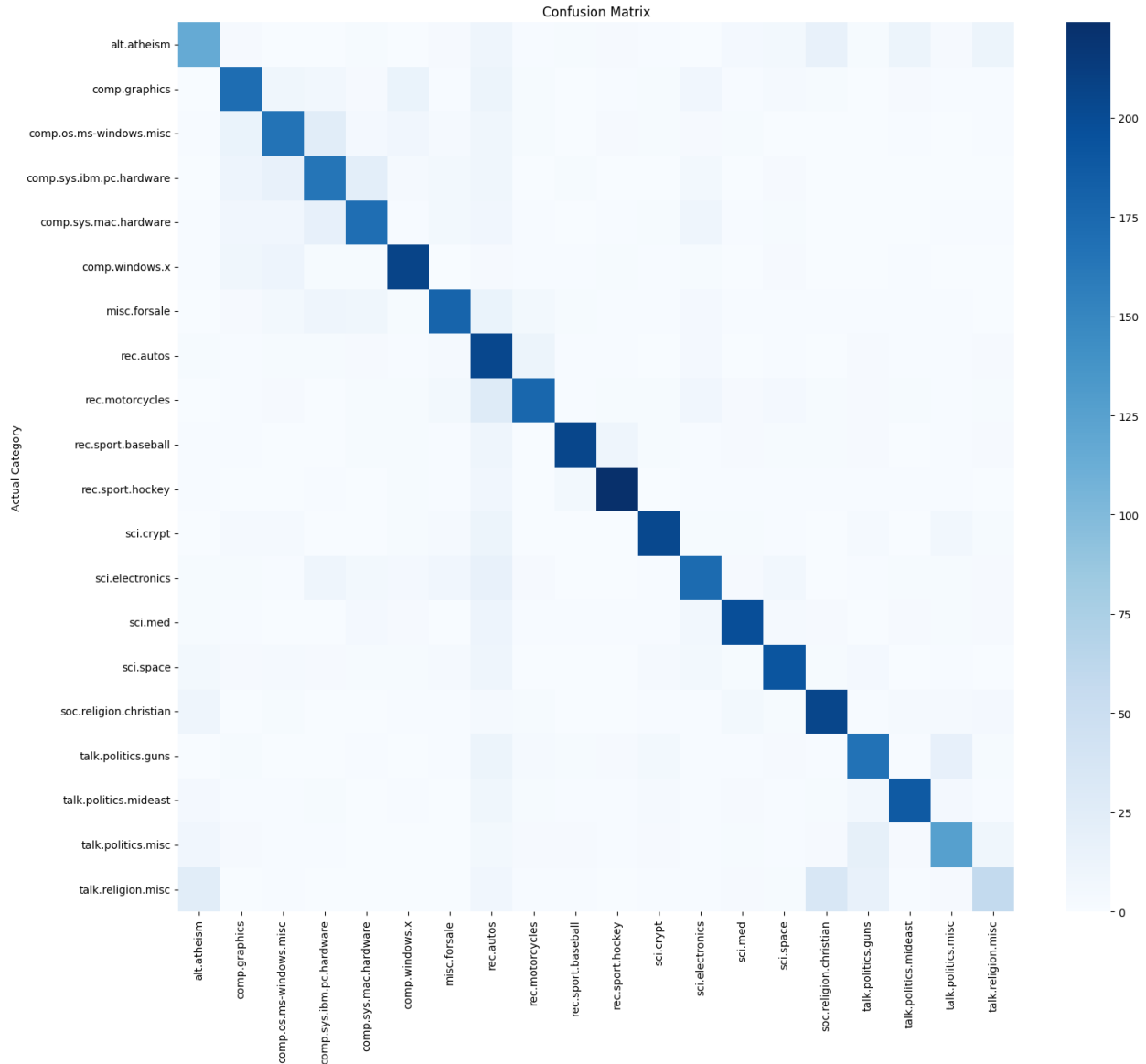


Figure 3: Confusion Matrix for Linear SVM Model

3.1 Model Analysis: Top Features per Category

Analyzing the coefficients of the trained Linear SVM model provides insight into which features (words or n-grams) the model found most important for classifying documents into each category. For each category,

the features with the largest positive coefficients are the most indicative of belonging to that category, while those with the largest negative coefficients are most indicative of *not* belonging to that category.

Based on the analysis, the top features per category were highly interpretable and relevant to the news-group topics. For example:

- For 'rec.autos', features like 'car', 'ford', 'dealer' had high positive coefficients, while terms related to motorcycles ('bike', 'ride') had negative coefficients.
- For 'rec.sport.hockey', terms like 'hockey', 'playoff', 'nhl' were highly positive.
- For 'sci.crypt', terms like 'encrypt', 'nsa', 'clipper' were strongly positive.
- For 'talk.politics.mideast', terms like 'israel', 'arab', 'armenian' were highly influential.

The analysis of negative features was also informative, highlighting terms that differentiate a category from others (e.g., terms strongly associated with one religion being negative features for another). The use of bigrams (like 'window manag', 'make offer') also appeared as influential features, demonstrating the value of considering word pairs.

4 Conclusion

This project successfully implemented a text classification pipeline to categorize news articles from the 20 Newsgroups dataset using NLP techniques. The process involved standard steps from data loading and analysis through preprocessing, feature extraction using TF-IDF with n-grams, and training a Linear SVM classifier.

The model achieved an overall accuracy of 0.7411, demonstrating its ability to classify documents significantly better than random chance on a challenging 20-class dataset. Evaluation metrics provided detailed insights into per-category performance, showing varying levels of classification success depending on the distinctiveness of the category's language. The analysis of model coefficients revealed highly interpretable features that align well with the expected topics of the newsgroups.

While the results are promising for a standard approach, potential areas for future work include:

- **Advanced Feature Representations:** Exploring more sophisticated techniques like word embeddings (e.g., Word2Vec, GloVe) or even early forms of contextual embeddings if feasible within the computational resources and knowledge context.
- **Alternative Models:** Experimenting with other classification algorithms (e.g., Logistic Regression, Naïve Bayes variants suited for TF-IDF, or potentially simple neural network architectures).
- **Hyperparameter Tuning:** Systematically optimizing parameters of the `TfidfVectorizer` and the `LinearSVC` using cross-validation.
- **Refined Preprocessing:** Investigating the impact of different stemming algorithms or lemmatization.

This project successfully demonstrated a fundamental and effective approach to text classification, providing a solid foundation for further exploration in NLP and Data Mining.

References

- Lang, K. (1995). Newsweeder: Learning to filter netnews. *Proceedings of the Twelfth International Conference on Machine Learning*, 331-339. (Source of the 20 Newsgroups dataset)
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- Bird, S., Klein, E., Loper, E. (2009). *Natural Language Processing with Python: Analyzing Text with the NLTK Book.* O'Reilly Media.